EX 5 Snort

Snort is a free, open-source Network Intrusion Detection System (NIDS) and Intrusion Prevention System (IPS) created in 1998 by Martin Roesch. It is one of the most widely used lightweight network security monitoring tools available.

Snort is a powerful open-source network intrusion detection and prevention system (NIDS/IPS) that was created in 1998 by Martin Roesch and has since become one of the most widely used security tools in the cybersecurity world. It is a lightweight, free software that monitors network traffic in real-time, analyzing packets to detect suspicious patterns, malicious activities, and potential threats based on signature-based detection, protocol analysis, and rule matching. Snort operates by capturing network packets and comparing them against a database of known attack signatures, allowing it to identify and alert administrators about potential security breaches such as malware, port scans, denial of service attacks, and other network vulnerabilities. The tool supports multiple operating systems including Linux and Windows, and provides flexibility through customizable rules that enable security professionals to create specific detection patterns tailored to their organization's needs. Snort uses a modular architecture that includes the packet decoder, detection engine, logging and alerting system, and supports various output plugins for different reporting formats. With an active community and regular rule updates from the official Snort website, it remains a critical component for network security monitoring, helping organizations detect and prevent emerging threats before they can cause significant damage. Whether deployed as a passive IDS or an active IPS, Snort provides essential visibility into network traffic and serves as a foundational tool for both individual security practitioners and enterprise security operations centers.

Key Features:

Real-time traffic analysis - Monitors network packets in real-time
Pattern matching - Detects suspicious patterns and potential threats using signature-based detection
Protocol detection - Analyzes protocols like TCP, UDP, ICMP
Flexible rules - Users can write custom rules to detect specific threats
Works on Linux and Windows
Community-driven - Has a large community contributing rules and updates
List all interfaces Snort can see: snort -W

```
PS C:\Snort> snort -W

   ,,_     -*> Snort! <*-
  o"  )~   Version 2.9.20-WIN64 GRE (Build 82)
   ''''    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
           Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
           Copyright (C) 1998-2013 Sourcefire, Inc., et al.
           Using PCRE version: 8.10 2010-06-25
           Using ZLIB version: 1.2.11

Index   Physical Address        IP Address      Device Name     Description
-----   ----------------        ----------      -----------     -----------
    1   00:00:00:00:00:00       disabled        \Device\NPF_{741353B7-7DDB-47FD-A0A5-D727FDEA6BB8}  WAN Miniport (Network Monitor)
    2   00:00:00:00:00:00       disabled        \Device\NPF_{3ECD7CD6-6F14-4E57-812F-A251683BCFF1}  WAN Miniport (IPv6)
    3   00:00:00:00:00:00       disabled        \Device\NPF_{63D946AD-0823-4BEE-9C25-1228DD847481}  WAN Miniport (IP)
    4   DC:4B:A1:1F:67:CC       192.168.43.26   \Device\NPF_{96CE75B6-09C8-479B-B34B-109D07789A3E}  Realtek RTL8851BE WiFi 6 802.11ax PCIe Adapter
    5   DE:4B:A1:1F:67:CC       169.254.98.136  \Device\NPF_{435F0042-62AE-4C05-B2E6-2CF08615E00D}  Realtek RTL8851BE WiFi 6 802.11ax PCIe Adapter #4
    6   D2:4B:A1:1F:67:CC       169.254.189.226 \Device\NPF_{4BEE1DA5-F4AA-47CD-941E-260DE6498559}  Realtek RTL8851BE WiFi 6 802.11ax PCIe Adapter #3
    7   00:00:00:00:00:00       0000:0000:0000:0000:0000:0000:0000:0000 \Device\NPF_Loopback        Adapter for loopback traffic capture
PS C:\Snort>
```

Figure 1.1 List all interfaces of Snort #run

snort in packet dump mode

snort -v -i "\Device\NPF_{741353B7-7DDB-47FD-A0A5-D727FDEA6BB8}"

```
PS C:\Snort> snort -v -i "\Device\NPF_{741353B7-7DDB-47FD-A0A5-D727FDEA6BB8}"
Running in packet dump mode

        --== Initializing Snort ==--
Initializing Output Plugins!
pcap DAQ configured to passive.
The DAQ version does not support reload.
Acquiring network traffic from "\Device\NPF_{741353B7-7DDB-47FD-A0A5-D727FDEA6BB8}".
Decoding Ethernet

        --== Initialization Complete ==--

   ,,_     -*> Snort! <*-
  o"  )~   Version 2.9.20-WIN64 GRE (Build 82)
   ''''    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
           Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
           Copyright (C) 1998-2013 Sourcefire, Inc., et al.
           Using PCRE version: 8.10 2010-06-25
           Using ZLIB version: 1.2.11

Commencing packet processing (pid=12912)
```

Figure 1.2 snort in packet dump mode

1. Read packets from the network interface and compare them to the set of rules specified in the configuration file.

Running in IDS mode

snort -i 4 -c C:\Snort\etc\snort.conf -l C:\Snort\log -A console

```
PS C:\Snort> snort -i  4 -c C:\Snort\etc\snort.conf -l C:\Snort\log -A console
Running in IDS mode

        --== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "C:\Snort\etc\snort.conf"
PortVar 'HTTP_PORTS' defined :  [ 80:81 311 383 591 593 901 1220 1414 1741 1830 2301 2381 2809 3037
777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888 8899 9
0 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined :  [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined :  [ 1024:65535 ]
PortVar 'SSH_PORTS' defined :  [ 22 ]
PortVar 'FTP_PORTS' defined :  [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined :  [ 5060:5061 5600 ]
         Preprocessor Object: SF_SDF  Version 1.1  <Build 1>
         Preprocessor Object: SF_REPUTATION  Version 1.1  <Build 1>
         Preprocessor Object: SF_POP  Version 1.0  <Build 1>
         Preprocessor Object: SF_MODBUS  Version 1.1  <Build 1>
         Preprocessor Object: SF_IMAP  Version 1.0  <Build 1>
         Preprocessor Object: SF_GTP  Version 1.1  <Build 1>
         Preprocessor Object: SF_FTPTELNET  Version 1.2  <Build 13>
         Preprocessor Object: SF_DNS  Version 1.1  <Build 4>
         Preprocessor Object: SF_DNP3  Version 1.1  <Build 1>
         Preprocessor Object: SF_DCERPC2  Version 1.0  <Build 3>
Commencing packet processing (pid=9748)
04/17-05:47:19.208975  [**] [1:1000001:1] ICMP Test - Snort is Working! [**] [Priority: 0] {IPV6-ICMP} fe80:0000:0000:0000:0215:5dff:fe5b:37f4 -> ff02:0000:
0000:0000:0000:0000:0000:0002
```

```
PS C:\Users\crsha> ping google.com

Pinging google.com [2404:6800:4009:808::200e] with 32 bytes of data:
Reply from 2404:6800:4009:808::200e: time=157ms
Reply from 2404:6800:4009:808::200e: time=164ms
Reply from 2404:6800:4009:808::200e: time=180ms
Reply from 2404:6800:4009:808::200e: time=82ms

Ping statistics for 2404:6800:4009:808::200e:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 82ms, Maximum = 180ms, Average = 145ms
```

Figure 1.3 Read packets from the network interface

2. Read packets from a PCAP file.

snort -r C:\Snort\log\snort.log.1776384683 -c C:\Snort\etc\snort.conf -l C:\Snort\log

-A console

```
PS C:\Snort> snort -r C:\Snort\log\snort.log.1776384683 -c C:\Snort\etc\snort.conf -l C:\Snort\log -A console
Running in IDS mode

        --== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "C:\Snort\etc\snort.conf"
PortVar 'HTTP_PORTS' defined :  [ 80:81 311 383 591 593 901 1220 1414 1741 1830 2301 2381 2809 3037 3128 3702 4343
777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888 8899 9000 9060 9080 9
0 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined :  [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined :  [ 1024:65535 ]
PortVar 'SSH_PORTS' defined :  [ 22 ]
PortVar 'FTP_PORTS' defined :  [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined :  [ 5060:5061 5600 ]
PortVar 'FILE_DATA_PORTS' defined :  [ 80:81 110 143 311 383 591 593 901 1220 1414 1741 1830 2301 2381 2809 3037 31
4:7145 7510 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888 8899 900
43:34444 41080 50002 55555 ]
PortVar 'GTP_PORTS' defined :  [ 2123 2152 3386 ]
Detection:
   Search-Method = AC-Full-Q
    Split Any/Any group = enabled
    Search-Method-Optimizations = enabled
    Maximum pattern length = 20
Tagged Packet Limit: 256
Loading dynamic engine C:\Snort\lib\snort_dynamicengine\sf_engine.dll... done
Loading all dynamic preprocessor libs from C:\Snort\lib\snort_dynamicpreprocessor...
  Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_dce2.dll... done
  Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_dnp3.dll... done
```

Figure 1.4 Read packets from a PCAP file.

3. Test Snort Configuration File snort -T -i 1 -c C:\Snort\etc\snort.conf

```
PS C:\Snort> snort -T -i 1 -c C:\Snort\etc\snort.conf
Running in Test mode

        --== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "C:\Snort\etc\snort.conf"
PortVar 'HTTP_PORTS' defined :  [ 80:81 311 383 591 593 901 1220 1414 1741 1
830 2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 75
10 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 824
3 8280 8300 8800 8888 8899 9000 9060 9080 9090:9091 9443 9999 11371 34443:34
444 41080 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined :  [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined :  [ 1024:65535 ]
PortVar 'SSH_PORTS' defined :  [ 22 ]
PortVar 'FTP_PORTS' defined :  [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined :  [ 5060:5061 5600 ]
```

```
            Preprocessor Object: SF_SIP  Version 1.1  <Build 1>
            Preprocessor Object: SF_SDF  Version 1.1  <Build 1>
            Preprocessor Object: SF_REPUTATION  Version 1.1  <Build 1>
            Preprocessor Object: SF_POP  Version 1.0  <Build 1>
            Preprocessor Object: SF_MODBUS  Version 1.1  <Build 1>
            Preprocessor Object: SF_IMAP  Version 1.0  <Build 1>
            Preprocessor Object: SF_GTP  Version 1.1  <Build 1>
            Preprocessor Object: SF_FTPTELNET  Version 1.2  <Build 13>
            Preprocessor Object: SF_DNS  Version 1.1  <Build 4>
            Preprocessor Object: SF_DNP3  Version 1.1  <Build 1>
            Preprocessor Object: SF_DCERPC2  Version 1.0  <Build 3>

 Total snort Fixed Memory Cost - MaxRss:-219860320
 Snort successfully validated the configuration!
 Snort exiting
 PS C:\Snort>
```

Figure 1.5 Test Snort Configuration File

4. Read packets from the configured network interface and write to a pcap file. snort -i 1 -c C:\Snort\etc\snort.conf -l C:\Snort\log -b

Figure 1.6 Read packets from the configured network interface and write to a pcap file.

```
PS C:\Snort> snort -i 4 -c C:\Snort\etc\snort.conf -l C:\Snort\log -b
Running in IDS mode

        --== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "C:\Snort\etc\snort.conf"
PortVar 'HTTP_PORTS' defined :  [ 80:81 311 383 591 593 901 1220 1414 1741
830 2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 7
10 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 82
3 8280 8300 8800 8888 8899 9000 9060 9080 9090:9091 9443 9999 11371 34443:3
444 41080 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined :  [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined :  [ 1024:65535 ]
PortVar 'SSH_PORTS' defined :  [ 22 ]
PortVar 'FTP_PORTS' defined :  [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined :  [ 5060:5061 5600 ]
```

5. create a simple rule that we can trigger at any time to test everything is working and to know how the alerting / logging works.

alert icmp any any -> any any (msg:"ICMP Test - Snort is Working!"; sid:1000001; rev:1;)

```
PS C:\Snort> snort -A console -q -c C:\Snort\etc\snort.conf -i 5
04/17-07:26:20.020606  [**] [1:1000021:1] ICMP Any Message to 8.8.8.8 [**] [
Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:26:20.020606  [**] [1:1000020:1] ICMP Traffic Directed to 8.8.8.8 (
Google DNS) [**] [Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:26:20.020606  [**] [1:1000001:1] ICMP Test - Snort is Working! [**]
 [Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:26:20.063845  [**] [1:1000001:1] ICMP Test - Snort is Working! [**]
 [Priority: 0] {ICMP} 8.8.8.8 -> 10.41.146.26
04/17-07:26:21.039808  [**] [1:1000021:1] ICMP Any Message to 8.8.8.8 [**] [
```

```
PS C:\Users\crsha> ping 127.0.0.1

Pinging 127.0.0.1 with 32 bytes of data:
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128

Ping statistics for 127.0.0.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
PS C:\Users\crsha> ping 127.0.0.1

Pinging 127.0.0.1 with 32 bytes of data:
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128

Ping statistics for 127.0.0.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
PS C:\Users\crsha>
```

Figure 1.7 trigger at any time to test everything is working on snort

6. print the alerts to the console

```
PS C:\Snort> snort -i 5 -c C:\Snort\etc\snort.conf -A console
Running in IDS mode

        --== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "C:\Snort\etc\snort.conf"
PortVar 'HTTP_PORTS' defined :  [ 80:81 311 383 591 593 901 1220 1414 174
830 2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145
10 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181
3 8280 8300 8800 8888 8899 9000 9060 9080 9090:9091 9443 9999 11371 34443
444 41080 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined :  [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined :  [ 1024:65535 ]
```

```
Commencing packet processing (pid=31352)
04/17-07:28:55.733367  [**] [1:1000021:1] ICMP Any Message to 8.8.8.8 [**] [
Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:28:55.733367  [**] [1:1000020:1] ICMP Traffic Directed to 8.8.8.8 (
Google DNS) [**] [Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:28:55.733367  [**] [1:1000001:1] ICMP Test - Snort is Working! [**]
 [Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:28:55.807874  [**] [1:1000001:1] ICMP Test - Snort is Working! [**]
 [Priority: 0] {ICMP} 8.8.8.8 -> 10.41.146.26
04/17-07:28:56.754139  [**] [1:1000021:1] ICMP Any Message to 8.8.8.8 [**] [
Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:28:56.754139  [**] [1:1000020:1] ICMP Traffic Directed to 8.8.8.8 (
Google DNS) [**] [Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:28:56.754139  [**] [1:1000001:1] ICMP Test - Snort is Working! [**]
 [Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:28:56.793342  [**] [1:1000001:1] ICMP Test - Snort is Working! [**]
 [Priority: 0] {ICMP} 8.8.8.8 -> 10.41.146.26
```

Figure 1.8 print the alerts to the console

7. limit the traffic that we are inspecting and capturing.

For ICMP traffic only:

snort -i 5 -c C:\Snort\etc\snort.conf -A console "icmp"

```
PS C:\Snort> snort -i 5 -c C:\Snort\etc\snort.conf -A console "icmp"
Running in IDS mode

        --== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "C:\Snort\etc\snort.conf"
PortVar 'HTTP_PORTS' defined :  [ 80:81 311 383 591 593 901 1220 1414 1741 1
830 2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 75
10 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 824
3 8280 8300 8800 8888 8899 9000 9060 9080 9090:9091 9443 9999 11371 34443:34
444 41080 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined :  [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined :  [ 1024:65535 ]
PortVar 'SSH_PORTS' defined :  [ 22 ]
PortVar 'FTP_PORTS' defined :  [ 21 2100 3535 ]
Commencing packet processing (pid=28124)
04/17-07:52:38.350092  [**] [1:1000021:1] ICMP Any Message to 8.8.8.8 [**] [
Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:52:38.350092  [**] [1:1000020:1] ICMP Traffic Directed to 8.8.8.8 (
Google DNS) [**] [Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:52:38.350092  [**] [1:1000001:1] ICMP Test - Snort is Working! [**]
 [Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:52:38.418993  [**] [1:1000001:1] ICMP Test - Snort is Working! [**]
 [Priority: 0] {ICMP} 8.8.8.8 -> 10.41.146.26
04/17-07:52:39.357709  [**] [1:1000021:1] ICMP Any Message to 8.8.8.8 [**] [
Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
```

Figure 1.9 limit the traffic that we are inspecting and capturing

8. Perform network scanning attacks with nmap #

SYN scan

nmap -sS 10.41.146.0/24

```
PS C:\Users\crsha> nmap -sS 10.41.146.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-17 08:21 +0530
Nmap scan report for 10.41.146.134
Host is up (0.0079s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE
53/tcp open  domain
MAC Address: 72:9C:21:5A:DC:CE (Unknown)

Nmap scan report for 10.41.146.26
Host is up (0.00035s latency).
Not shown: 997 closed tcp ports (reset)
PORT    STATE SERVICE
135/tcp open  msrpc
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds

Nmap done: 256 IP addresses (2 hosts up) scanned in 40.76 seconds
PS C:\Users\crsha>
```

Figure 1.10 SYN scan

UDP scan

nmap -sU 192.18.43.0

```
PS C:\Users\crsha> nmap -sU --top-ports 20 10.41.146.26
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-17 08:36 +0530
Nmap scan report for 10.41.146.26
Host is up (0.00088s latency).

PORT      STATE         SERVICE
53/udp    closed        domain
67/udp    closed        dhcps
68/udp    closed        dhcpc
69/udp    closed        tftp
123/udp   closed        ntp
135/udp   closed        msrpc
137/udp   open|filtered netbios-ns
138/udp   open|filtered netbios-dgm
139/udp   closed        netbios-ssn
161/udp   closed        snmp
162/udp   closed        snmptrap
445/udp   closed        microsoft-ds
500/udp   open|filtered isakmp
514/udp   closed        syslog
520/udp   closed        route
631/udp   closed        ipp
1434/udp  closed        ms-sql-m
1900/udp  open|filtered upnp
4500/udp  open|filtered nat-t-ike
49152/udp closed        unknown

Nmap done: 1 IP address (1 host up) scanned in 2.31 seconds
PS C:\Users\crsha>
```

Figure 1.11 UDP scan

FIN scan

nmap -sF 192.168.43.0/24

```
PS C:\Users\crsha> nmap -sF 10.41.146.26
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-17 08:36 +0530
Nmap scan report for 10.41.146.26
Host is up (0.00085s latency).
All 1000 scanned ports on 10.41.146.26 are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 1.30 seconds
PS C:\Users\crsha>
```

Figure 1.12 FIN scan

NULL scan
nmap -sN 192.168.43.0/24

```
PS C:\Users\crsha> nmap -sN 10.152.126.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-18 21:49 +0530
Nmap scan report for 10.152.126.9
Host is up (0.0090s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE          SERVICE
53/tcp open|filtered domain
MAC Address: 72:9C:21:5A:DC:CE (Unknown)

Nmap scan report for 10.152.126.26
Host is up (0.00016s latency).
All 1000 scanned ports on 10.152.126.26 are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 256 IP addresses (2 hosts up) scanned in 5.37 seconds
```

Figure 1.13 NULL scan

XMAS scan
nmap -sX 192.168.43.0/24

```
PS C:\Users\crsha> nmap -sX 10.152.126.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-18 21:50 +0530
Nmap scan report for 10.152.126.9
Host is up (0.0082s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE          SERVICE
53/tcp open|filtered domain
MAC Address: 72:9C:21:5A:DC:CE (Unknown)

Nmap scan report for 10.152.126.26
Host is up (0.00021s latency).
All 1000 scanned ports on 10.152.126.26 are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 256 IP addresses (2 hosts up) scanned in 5.58 seconds
```

Figure 1.14 XMAS scan

PING scan
nmap -sP 192.168.43.0/24

```
PS C:\Users\crsha> nmap -sP 10.152.126.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-18 21:50 +0530
Nmap scan report for 10.152.126.9
Host is up (0.014s latency).
MAC Address: 72:9C:21:5A:DC:CE (Unknown)
Nmap scan report for 10.152.126.26
Host is up.
Nmap done: 256 IP addresses (2 hosts up) scanned in 3.42 seconds
```

Figure 1.15 PING scan

TCP connect scan nmap -
sT 192.168.43.0/24

```
PS C:\Users\crsha> nmap -sT 10.152.126.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-18 21:51 +0530
Nmap scan report for 10.152.126.9
Host is up (0.013s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT    STATE    SERVICE
53/tcp  open     domain
110/tcp filtered pop3
139/tcp filtered netbios-ssn
MAC Address: 72:9C:21:5A:DC:CE (Unknown)

Nmap scan report for 10.152.126.26
Host is up (0.0020s latency).
Not shown: 996 closed tcp ports (conn-refused)
PORT     STATE SERVICE
135/tcp  open  msrpc
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
5357/tcp open  wsdapi

Nmap done: 256 IP addresses (2 hosts up) scanned in 30.97 seconds
```

Figure 1.16 TCP scan

OS detection + version + scripts nmap -A
192.168.43.0/24

```
PS C:\Users\crsha> nmap -A 10.152.126.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-18 21:52 +0530
Nmap scan report for 10.152.126.9
Host is up (0.0039s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
53/tcp open  domain  dnsmasq 2.51
| dns-nsid:
|_  bind.version: dnsmasq-2.51
MAC Address: 72:9C:21:5A:DC:CE (Unknown)
Device type: phone
Running: Google Android 9.X|10.X|11.X, Linux 4.X
OS CPE: cpe:/o:google:android:9 cpe:/o:google:android:10 cpe:/o:google:android:11 cpe:/o:linux:linux_kernel:4
OS details: Android 9 - 11 (Linux 4.9 - 4.14)
Network Distance: 1 hop

TRACEROUTE
HOP RTT     ADDRESS
1   3.87 ms 10.152.126.9

Nmap scan report for 10.152.126.26
Host is up (0.038s latency).
Not shown: 996 closed tcp ports (reset)
PORT     STATE SERVICE       VERSION
135/tcp  open  msrpc         Microsoft Windows RPC
139/tcp  open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp  open  microsoft-ds?
5357/tcp open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-title: Service Unavailable
|_http-server-header: Microsoft-HTTPAPI/2.0
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.99%E=4%D=4/18%OT=135%CT=1%CU=43780%PV=Y%DS=0%DC=L%G=Y%TM=69E3B0
OS:32%P=i686-pc-windows-windows)SEQ(SP=103%GCD=1%ISR=10E%TI=I%CI=I%TS=A)SEQ
OS:(SP=107%GCD=1%ISR=106%TI=I%CI=I%TS=A)SEQ(SP=107%GCD=1%ISR=10B%TI=I%CI=I%
OS:TS=A)SEQ(SP=107%GCD=1%ISR=10C%TI=I%CI=I%TS=A)SEQ(SP=FE%GCD=1%ISR=10A%TI=
OS:I%CI=I%TS=A)OPS(O1=MFFD7NW8ST11%O2=MFFD7NW8ST11%O3=MFFD7NW8NNT11%O4=MFFD
OS:7NW8ST11%O5=MFFD7NW8ST11%O6=MFFD7ST11)WIN(W1=FFFF%W2=FFFF%W3=FFFF%W4=FFF
OS:F%W5=FFFF%W6=FFFF)ECN(R=Y%DF=Y%T=80%W=FFFF%O=MFFD7NW8NNS%CC=N%Q=)T1(R=Y%
OS:DF=Y%T=80%S=O%A=S+%F=AS%RD=0%Q=)T2(R=Y%DF=Y%T=80%W=0%S=Z%A=S%F=AR%O=%RD=
OS:0%Q=)T3(R=Y%DF=Y%T=80%W=0%S=Z%A=O%F=AR%O=%RD=0%Q=)T4(R=Y%DF=Y%T=80%W=0%S
OS:=A%A=O%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=80%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)T6(R
OS:=Y%DF=Y%T=80%W=0%S=A%A=O%F=R%O=%RD=0%Q=)T7(R=Y%DF=Y%T=80%W=0%S=Z%A=S+%F=
OS:AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=80%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%
OS:RUD=G)IE(R=N)

Network Distance: 0 hops
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
| smb2-time:
|   date: 2026-04-18T16:24:05
|_  start_date: N/A
| smb2-security-mode:
|   3.1.1:
|_    Message signing enabled and required

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 256 IP addresses (2 hosts up) scanned in 93.84 seconds
```

Figure 1.17 scan OS detection -version - scripts

G. Add rules to detect SYN attack, UDP scan, PINK scan, FIN scan, NULL scan, XMAS scan, and TCP scan.

SYN scan
alert tcp any any -> $HOME_NET any (msg:"SCAN SYN Scan Detected"; flags:S,12; threshold: type both, track by_src, count 5, seconds 2; sid:1000010; rev:1;)
UDP scan
alert udp any any -> $HOME_NET any (msg:"SCAN UDP Scan Detected"; threshold: type both, track by_src, count 10, seconds 2; sid:1000011; rev:1;)
FIN scan
alert tcp any any -> $HOME_NET any (msg:"SCAN FIN Scan Detected"; flags:F,12; threshold: type both, track by_src, count 5, seconds 2; sid:1000012; rev:1;)
NULL scan
alert tcp any any -> $HOME_NET any (msg:"SCAN NULL Scan Detected"; flags:0; threshold: type both, track by_src, count 5, seconds 2; sid:1000013; rev:1;)
XMAS scan
alert tcp any any -> $HOME_NET any (msg:"SCAN XMAS Scan Detected"; flags:FPU,12; threshold: type both, track by_src, count 5, seconds 2; sid:1000014; rev:1;)
PING/ICMP scan
alert icmp any any -> $HOME_NET any (msg:"SCAN ICMP Ping Sweep Detected"; itype:8; threshold: type both, track by_src, count 5, seconds 2; sid:1000015; rev:1;) # TCP connect scan

alert tcp any any -> $HOME_NET any (msg:"SCAN TCP Connect Scan Detected"; flags:SA; threshold: type both, track by_src, count 10, seconds 2; sid:1000016; rev:1;)

10. Execute Snort in IDS mode

```
PS C:\Snort> snort -i  4 -c C:\Snort\etc\snort.conf -l C:\Snort\log -A console
Running in IDS mode

        --== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "C:\Snort\etc\snort.conf"
PortVar 'HTTP_PORTS' defined :  [ 80:81 311 383 591 593 901 1220 1414 1741 1830 2301 2381 2809 3037
777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888 8899 9
0 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined :  [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined :  [ 1024:65535 ]
PortVar 'SSH_PORTS' defined :  [ 22 ]
PortVar 'FTP_PORTS' defined :  [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined :  [ 5060:5061 5600 ]
```

Figure 1.18 Execute Snort in IDS mode

11. Write a rule to alert whenever ICMP traffic is sent to the IP address 8.8.8.8 and for detecting ping requests or any other ICMP messages directed at that specific server.

alert icmp any any -> 8.8.8.8 any (msg:"ICMP Traffic Directed to 8.8.8.8 (Google DNS)"; itype:8; sid:1000020; rev:1;)

alert icmp any any -> 8.8.8.8 any (msg:"ICMP Any Message to 8.8.8.8"; sid:1000021; rev:1;)

```
Commencing packet processing (pid=31352)
04/17-07:28:55.733367  [**] [1:1000021:1] ICMP Any Message to 8.8.8.8 [**] [
Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:28:55.733367  [**] [1:1000020:1] ICMP Traffic Directed to 8.8.8.8 (
Google DNS) [**] [Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:28:55.733367  [**] [1:1000001:1] ICMP Test - Snort is Working! [**]
 [Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:28:55.807874  [**] [1:1000001:1] ICMP Test - Snort is Working! [**]
 [Priority: 0] {ICMP} 8.8.8.8 -> 10.41.146.26
04/17-07:28:56.754139  [**] [1:1000021:1] ICMP Any Message to 8.8.8.8 [**] [
Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:28:56.754139  [**] [1:1000020:1] ICMP Traffic Directed to 8.8.8.8 (
Google DNS) [**] [Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:28:56.754139  [**] [1:1000001:1] ICMP Test - Snort is Working! [**]
 [Priority: 0] {ICMP} 10.41.146.26 -> 8.8.8.8
04/17-07:28:56.793342  [**] [1:1000001:1] ICMP Test - Snort is Working! [**]
 [Priority: 0] {ICMP} 8.8.8.8 -> 10.41.146.26
```

Figure 1.19 detecting ping requests

12. Write a rule to Detect a Simple HTTP GET Request to a Certain Domain.

alert tcp any any -> any 8080 (msg:"HTTP GET Request to example.com"; content:"example.com"; nocase; sid:1000030; rev:1;)

```
PS C:\Users\crsha> curl http://example.com

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the
web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.
```

```
Commencing packet processing (pid=13392)
04/19-07:13:51.208901  [**] [1:1000030:1] HTTP GET Request to example.com [*
*] [Priority: 0] {TCP} 2401:4900:258e:c1bd:18e3:85be:9620:08c9:5882 -> 2606:
4700:83b5:72db:f23b:0005:ef6b:ff98:80
04/19-07:14:00.579994  [**] [1:1000001:1] ICMP Test - Snort is Working! [**]
 [Priority: 0] {IPV6-ICMP} fe80:0000:0000:0000:709c:21ff:fe5a:dcce -> 2401:4
900:258e:c1bd:18e3:85be:9620:08c9
```

Figure 1.20 Detect a Simple HTTP GET Request to a Certain Domain

13. Write a rule to Detect a Suspicious User-Agent String.

alert tcp any any -> any 8080 (msg:"Suspicious User-Agent - curl"; content:"curl"; nocase; sid:1000040; rev:1;)

alert tcp any any -> any 8080 (msg:"Suspicious User-Agent - sqlmap"; content:"sqlmap"; nocase; sid:1000041; rev:1;)

alert tcp any any -> any 8080 (msg:"Suspicious User-Agent - Nikto"; content:"Nikto"; nocase; sid:1000042; rev:1;)

```
PS C:\Users\crsha> curl.exe -A "curl/7.68.0" http://example.com
<!doctype html><html lang="en"><head><title>Example Domain</title><meta name
="viewport" content="width=device-width, initial-scale=1"><style>body{backgr
ound:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{fo
nt-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><bo
dy><div><h1>Example Domain</h1><p>This domain is for use in documentation ex
amples without needing permission. Avoid use in operations.</p><p><a href="h
ttps://iana.org/domains/example">Learn more</a></p></div></body></html>
04/19-07:16:35.815356  [**] [1:1000030:1] HTTP GET Request to example.com [*
*] [Priority: 0] {TCP} 2401:4900:258e:c1bd:18e3:85be:9620:08c9:16064 -> 2606
:4700:83b5:72db:f2a5:0005:ef6b:ff98:80
04/19-07:16:35.815356  [**] [1:1000040:1] Suspicious User-Agent - curl [**]
[Priority: 0] {TCP} 2401:4900:258e:c1bd:18e3:85be:9620:08c9:16064 -> 2606:47
00:83b5:72db:f2a5:0005:ef6b:ff98:80
04/19-07:16:41.139438  [**] [1:1000001:1] ICMP Test - Snort is Working! [**]
 [Priority: 0] {IPV6-ICMP} fe80:0000:0000:0000:709c:21ff:fe5a:dcce -> 2401:4
900:258e:c1bd:18e3:85be:9620:08c9
```

Figure 1.21 Detect a Suspicious User-Agent String-curl

```
PS C:\Users\crsha> curl.exe -A "sqlmap/1.4.7" http://example.com
<!doctype html><html lang="en"><head><title>Example Domain</title><meta name
="viewport" content="width=device-width, initial-scale=1"><style>body{backgr
ound:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{fo
nt-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><bo
dy><div><h1>Example Domain</h1><p>This domain is for use in documentation ex
amples without needing permission. Avoid use in operations.</p><p><a href="h
ttps://iana.org/domains/example">Learn more</a></p></div></body></html>
04/19-07:16:48.414126  [**] [1:1000030:1] HTTP GET Request to example.com [*
*] [Priority: 0] {TCP} 2401:4900:258e:c1bd:18e3:85be:9620:08c9:16068 -> 2606
:4700:83b5:72db:f2a5:0005:ef6b:ff98:80
04/19-07:16:48.414126  [**] [1:1000041:1] Suspicious User-Agent - sqlmap [**
] [Priority: 0] {TCP} 2401:4900:258e:c1bd:18e3:85be:9620:08c9:16068 -> 2606:
4700:83b5:72db:f2a5:0005:ef6b:ff98:80
```

Figure 1.22 Detect a Suspicious User-Agent String.-sqlmap

```
PS C:\Users\crsha> curl.exe -A "Nikto/2.1.6" http://example.com
<!doctype html><html lang="en"><head><title>Example Domain</title><meta name
="viewport" content="width=device-width, initial-scale=1"><style>body{backgr
ound:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{fo
nt-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><bo
dy><div><h1>Example Domain</h1><p>This domain is for use in documentation ex
amples without needing permission. Avoid use in operations.</p><p><a href="h
ttps://iana.org/domains/example">Learn more</a></p></div></body></html>
PS C:\Users\crsha>
04/19-07:17:01.284213  [**] [1:1000042:1] Suspicious User-Agent - Nikto Scan
ner [**] [Priority: 0] {TCP} 2401:4900:258e:c1bd:18e3:85be:9620:08c9:16071 -
> 2606:4700:83b5:72db:f2fd:0623:ef6b:ff98:80
04/19-07:17:05.841989  [**] [1:1000001:1] ICMP Test - Snort is Working! [**]
[Priority: 0] {IPV6-ICMP} 2401:4900:258e:c1bd:18e3:85be:9620:08c9 -> 2401:4
900:258e:c1bd:0000:0000:0000:00ec
```

Figure 1.23 Detect a Suspicious User-Agent String-Nikto

14. Write a Rule to Detect a Specific Vulnerability in a Web Application.

alert tcp $EXTERNAL_NET alert tcp any any -> any 8080 (msg:"SQL Injection OR 1=1 Detected"; content:"OR"; nocase; content:"1=1"; nocase; sid:1000050; rev:1;)

alert tcp any any -> any 8080 (msg:"SQL Injection UNION SELECT Detected"; content:"UNION"; nocase; content:"SELECT"; nocase; sid:1000051; rev:1;)

```
PS C:\Users\crsha> python -m http.server 8080
Serving HTTP on :: port 8080 (http://[::]:8080/) ...
::ffff:10.152.126.26 - - [19/Apr/2026 07:31:45] "GET /?host=example.com HTTP
/1.1" 200 -
::ffff:10.152.126.26 - - [19/Apr/2026 07:33:18] "GET /?host=example.com HTTP
/1.1" 200 -
::ffff:127.0.0.1 - - [19/Apr/2026 07:34:49] "GET / HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [19/Apr/2026 07:35:10] "GET /?id=1%20OR%201=1 HTTP/1.1"
 200 -
::ffff:127.0.0.1 - - [19/Apr/2026 07:35:20] "GET /?id=1%20UNION%20SELECT%201
,2,3 HTTP/1.1" 200 -
```

```
PS C:\Users\crsha> curl.exe "http://127.0.0.1:8080/?id=1%20OR%201=1"
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
04/19-07:35:10.545143  [**] [1:1000040:1] Suspicious User-Agent - curl [**]
[Priority: 0] {TCP} 127.0.0.1:8663 -> 127.0.0.1:8080
04/19-07:35:10.545143  [**] [1:1000050:1] SQL Injection OR 1=1 Detected [**]
[Priority: 0] {TCP} 127.0.0.1:8663 -> 127.0.0.1:8080
```

Figure 1.24 Detect a Specific Vulnerability in a Web Application- SQL Injection using OR

```
PS C:\Users\crsha> curl.exe "http://127.0.0.1:8080/?id=1%20UNION%20SELECT%20
1,2,3"
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
```

```
04/19-07:35:20.119818  [**] [1:1000051:1] SQL Injection UNION SELECT Detecte
d [**] [Priority: 0] {TCP} 127.0.0.1:8665 -> 127.0.0.1:8080
04/19-07:35:20.119818  [**] [1:1000040:1] Suspicious User-Agent - curl [**]
[Priority: 0] {TCP} 127.0.0.1:8665 -> 127.0.0.1:8080
```

Figure 1.25 Detect a Specific Vulnerability in a Web Application-SQL Injection using UNION

15. Write a Rule to Detect a Suspicious DNS Query.

alert udp any any -> any 53 (msg:"Suspicious DNS Query - .tk domain Detected"; content:"|02|tk|00|"; sid:1000060; rev:1;)

alert udp any any -> any 53 (msg:"Suspicious DNS Query - malware keyword Detected"; content:"malware"; nocase; sid:1000061; rev:1;)

```
PS C:\Users\crsha> nslookup example.tk 8.8.8.8
Server:  dns.google
Address:  8.8.8.8
```

```
Commencing packet processing (pid=5796)
04/19-07:43:46.929130  [**] [1:1000060:1] Suspicious DNS Query - .tk domain
[**] [Priority: 0] {UDP} 10.152.126.26:52153 -> 8.8.8.8:53
04/19-07:43:47.233356  [**] [1:1000060:1] Suspicious DNS Query - .tk domain
[**] [Priority: 0] {UDP} 10.152.126.26:52154 -> 8.8.8.8:53
04/19-07:43:47.547886  [**] [1:1000060:1] Suspicious DNS Query - .tk domain
```

Figure 1.26 Detect a Suspicious DNS Query-.tk domain

```
PS C:\Users\crsha> nslookup malware-test.com 8.8.8.8
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
Name:    malware-test.com
Address:  185.53.179.128
Commencing packet processing (pid=1836)
04/19-07:48:12.754269  [**] [1:1000061:1] Suspicious DNS Query - malware key
word [**] [Priority: 0] {UDP} 10.152.126.26:56026 -> 8.8.8.8:53
04/19-07:48:12.859747  [**] [1:1000061:1] Suspicious DNS Query - malware key
word [**] [Priority: 0] {UDP} 10.152.126.26:56027 -> 8.8.8.8:53
```

Figure 1.27 Detect a Suspicious DNS Query- malware keyword